

UNIVERSIDAD
NACIONAL
DE COLOMBIA

Sistema de Gestión de Inventarios - AgroInsumos Cafeteros Ltda.

Proyecto Ingeniería de Software II

Jean Byron Duque Carmona
Sebastián Gutiérrez Pineda

Universidad Nacional de Colombia
Facultad de Administración
Departamento de Informática y Computación
Manizales, Colombia
2026

Definición de problema y alcance.

AgroInsumos Cafeteros Ltda.

La empresa

AgroInsumos Cafeteros es una distribuidora familiar fundada en 2004 en Chinchiná, Caldas. Vende fertilizantes, semillas, agroquímicos, herramienta y equipo menor a caficultores pequeños y medianos, a fincas grandes y a algunas cooperativas del Eje Cafetero. Tiene unos 45 empleados.

Sedes y bodegas

Sede	**Tipo**	**Función**
Chinchiná	Bodega central + oficina administrativa	Recibe casi todo lo que llega de los proveedores y abastece a las demás.
Manizales	Punto de venta con bodega	La sede urban con más volumen.
Santa Rosa de Cabal	Punto de venta con bodega	Atiende la zona limitrofe con Risaralda.
Anserma	Bodega rural	Atiende el Occidente de Caldas.
Riosucio	Bodega rural	Es la más pequeña y alejada, con conectividad limitada.

Productos: unas 1.500 referencias activas, repartidas aproximadamente así:

- Fertilizantes (urea, DAP, compuestos, enmiendas): ~250 referencias. Son de alto volumen y se venden por bulto.
- Agroquímicos (fungicidas contra la roya, insecticidas contra la broca, herbicidas): ~450. Tienen lote y fecha de vencimiento.
- Semillas y material vegetal: ~100. También tienen lote y vencimiento.
- Herramienta y equipo (fumigadoras, machetes, despulpadoras pequeñas, lonas): ~500.
- Repuestos y accesorios de equipo: ~200.

Proveedores: unos 20 habituales, entre multinacionales de agroquímicos, importadores de fertilizantes, cooperativas y fabricantes nacionales de herramienta. Emiten entre 40 y 60 órdenes de compra al mes, y en temporada de fertilización llegan a más de 100.

Usuarios y roles

Unas 25 personas usarían el sistema.

Rol	Cantidad	Qué hace
Administrador del sistema	1	Gestiona usuarios, roles y parámetros generales
Gerente general	1	Consulta existencias consolidadas y reportes; aprueba compras grandes
Jefe de compras	1	Gestiona proveedores y crea las órdenes de compra
Auxiliar de compras	1	Hace seguimiento a las órdenes y registra recepciones con el bodeguero
Jefe de bodega central	1	Aprueba y despacha traslados; controla lotes y vencimientos
Bodeguero	6–8	Registra entradas, salidas, recepciones de traslado y conteos
Vendedor de punto	8–10	Consulta disponibilidad en todas las sedes y solicita traslados
Auditor / contador	1	Solo lectura, para consultar movimientos y devoluciones

Problemas concretos

1. **Desabastecimiento en temporada.** En el pico de fertilización, Anserma y Riosucio se quedan sin urea ni fungicida para la roya mientras Chinchiná tiene cientos de bultos. Como nadie ve el stock consolidado en tiempo real, el traslado se pide tarde y se pierden ventas cuando el caficultor más lo necesita.
2. **Pérdidas por vencimiento.** Se venden primero los agroquímicos que llegaron de último, porque no se controla qué lote vence antes, y cada año se botan o se devuelven con descuento productos vencidos. Además, el proveedor muchas veces rechaza la devolución porque no hay soporte de cuándo y en qué lote entró el producto.
3. **Traslados que se pierden o se duplican.** La central descuenta la mercancía, pero la sede destino no la suma, o la suma dos veces. En los conteos físicos aparecen diferencias que nadie puede explicar, y no se sabe si hubo robo, error de registro o mercancía en tránsito.

4. **Compras a ciegas.** Sin stock mínimo ni historial de consumo por sede, se compra de más en referencias de baja rotación y de menos en las críticas. Tampoco es fácil saber qué órdenes están pendientes, cuáles llegaron incompletas o qué proveedor incumple los tiempos de entrega.
5. **Cero trazabilidad en devoluciones.** Cuando un cliente reporta un lote defectuoso, la empresa no puede decir a qué otras sedes y clientes llegó ese mismo lote, ni reclamarle al proveedor con evidencia.

Alcance del sistema

Qué incluye

Catálogo de productos

- Referencias con categoría, unidad de medida (bulto, litro, unidad), precio de referencia y proveedor habitual.
- Indicador de si el producto requiere control por lote y vencimiento.

Inventario por bodega

- Existencias por producto, bodega y lote, con consulta consolidada de todas las sedes.
- Movimientos de entrada, salida y ajuste, cada uno con usuario, fecha y motivo.
- Stock mínimo por producto y bodega, con alertas de bajo stock y de lotes próximos a vencer.
- Sugerencia de salida por lote según el criterio FEFO: sale primero lo que vence primero.

Traslados entre bodegas

- Flujo completo: solicitud → aprobación → despacho → en tránsito → recepción.
- Registro de diferencias entre lo que se despachó y lo que se recibió.
- Descuento en origen y suma en destino coordinados entre servicios, con compensación si algún paso falla.

Proveedores y órdenes de compra

- Directorio de proveedores.
- Órdenes de compra con estados: borrador, aprobada, enviada, recibida parcialmente, recibida y cancelada.
- Recepción contra la orden, registrando lote y vencimiento, que genera la entrada automática al inventario.

Devoluciones

- **A proveedor:** por vencimiento, daño o defecto, con lote, cantidad, motivo, evidencias (fotos y observaciones) y estados (solicitada, enviada, aceptada o rechazada).
- **De cliente:** registrada como entrada con motivo y un número de factura externo como referencia, sin módulo de ventas.
- Trazabilidad por lote: en qué bodegas estuvo y qué movimientos tuvo.

Transversal

- API Gateway con autenticación, autorización por rol, versionado de API y documentación.
- Reportes básicos: existencias consolidadas, movimientos por período, lotes por vencer, órdenes pendientes y devoluciones.
- Bitácora de auditoría de movimientos.
- Despliegue en Kubernetes, con Docker Compose para el desarrollo y CI/CD con Jenkins, como pide el curso.

Qué queda fuera

Fuera del alcance	**Motivo o simplificación**
Ventas, POS y facturación electrónica DIAN	Las salidas por venta se registran como movimientos manuales
Contabilidad, cartera y pagos a proveedores	La orden de compra llega hasta la recepción, no hasta el pago
Crédito a caficultores	Es otro dominio, el financiero
Nómina y recursos humanos	No tiene relación con el inventario
Rutas, vehículos y logística de transporte	Del traslado solo interesa el estado "en tránsito", no el camión
Tienda en línea o portal para clientes	Solo hay usuarios internos
App móvil nativa y modo sin conexión para Riosucio	Se deja como trabajo futuro y se menciona como riesgo
Pronóstico de demanda con IA o ML	Solo alertas por stock mínimo fijo
Cumplimiento regulatorio ICA (registros, almacenamiento separado por categoría toxicológica)	El registro ICA queda como un campo informativo del catálogo, sin validaciones
Integración con ERP u otros sistemas externos	Sistema autónomo
Mensajería asíncrona (colas y eventos)	Decisión del equipo: comunicación solo REST

Repositorio y control de versiones

Repositorio del proyecto

- Nombre: ProyectoIngSoftware2
- Plataforma: GitHub

- Propósito: centralizar el código fuente y los recursos del proyecto.
- URL: Repositorio ProyectoIngSoftware2

Investigación de sistemas similares

Para orientar el diseño del sistema de inventarios de AgroInsumos Cafeteros Ltda. se analizaron tres soluciones de referencia: Odoo Inventory, Zoho Inventory y ERPNext. Los datos proceden de la documentación y las páginas oficiales de cada producto, consultadas en septiembre de 2026. Las referencias numeradas remiten a la lista final.

Requerimientos funcionales

Los requerimientos funcionales describen qué debe hacer el sistema.

ID	Requerimiento funcional	Descripción
RF01	Gestión de usuarios y roles	Permitir administrar usuarios, roles y permisos según las funciones de cada integrante.
RF02	Autenticación	Permitir el inicio de sesión de los usuarios y controlar el acceso al sistema.
RF03	Recuperación de contraseña	Permitir recuperar el acceso a una cuenta mediante un mecanismo de recuperación de contraseña.
RF04	Gestión de productos / catálogo	Registrar y consultar productos con categoría, unidad de medida, precio de referencia, proveedor habitual y características de control por lote/vencimiento.
RF05	Gestión de inventario por bodega	Consultar existencias por producto, bodega y lote, incluyendo una vista consolidada de las diferentes sedes.
RF06	Registro de movimientos	Registrar entradas, salidas y ajustes de inventario, asociándolos con usuario, fecha y motivo.
RF07	Control de lotes y vencimientos	Registrar lotes y fechas de vencimiento y permitir consultar su trazabilidad.

RF08	FEFO	Sugerir automáticamente la salida del lote que tenga la fecha de vencimiento más próxima (*First Expired, First Out*).
RF09	Traslados entre bodegas	Gestionar el ciclo solicitud → aprobación → despacho → en tránsito → recepción, incluyendo diferencias entre cantidades despachadas y recibidas.
RF10	Gestión de proveedores	Registrar y consultar la información de los proveedores de la empresa.
RF11	Órdenes de compra y recepción	Crear órdenes de compra, controlar sus estados y registrar recepciones completas o parciales, incluyendo lote y vencimiento.
RF12	Devoluciones	Gestionar devoluciones a proveedores y devoluciones de clientes, conservando información del lote, cantidad, motivo y estado.
RF13	Alertas	Generar alertas por stock mínimo y por productos/lotes próximos a vencer.
RF14	Reportes y dashboard	Presentar información consolidada mediante dashboard y reportes de existencias, movimientos, vencimientos, órdenes pendientes, devoluciones, valorización y rotación.

Requerimientos no funcionales

Los no funcionales establecen cómo debe funcionar el sistema, es decir, sus características de calidad, restricciones técnicas y condiciones de operación.

En el documento aparecen principalmente estos:

ID	**Requerimiento no funcional**	**Especificación**
RNF01	Seguridad	El acceso debe estar protegido mediante autenticación y autorización por roles. El Gateway valida la firma y

		vigencia del JWT y los servicios aplican autorización según el rol.
RNF02	Aislamiento de servicios	Los microservicios deben ejecutarse independientemente y cada uno debe mantener su propia base de datos.
RNF03	Disponibilidad / tolerancia a fallos	La caída de funciones secundarias como reportes o alertas no debe detener las operaciones críticas de inventario.
RNF04	Consistencia	Las operaciones críticas deben evitar duplicaciones y mantener consistencia mediante transacciones locales, compensaciones e identificadores de idempotencia.
RNF05	Rendimiento	Las operaciones críticas deben mantenerse ligeras y las consultas pesadas de reportes no deben competir directamente con las operaciones transaccionales.
RNF06	Escalabilidad	Los componentes deben poder escalar independientemente cuando aumente la carga de trabajo.
RNF07	Mantenibilidad	Los dominios deben mantenerse separados mediante microservicios y contratos REST claramente definidos.
RNF08	Auditoría / trazabilidad	Los movimientos deben conservar información del usuario, fecha y motivo, permitiendo reconstruir el historial de las operaciones.
RNF09	Interoperabilidad	La comunicación entre los servicios se realizará mediante APIs REST y un API Gateway.
RNF10	Despliegue	El sistema se plantea para ejecutarse mediante Docker/Kubernetes, utilizando Docker Compose durante el desarrollo y Kubernetes para el despliegue.

RNF11	Integración continua	El proyecto contempla CI/CD mediante Jenkins.
RNF12	Resiliencia de comunicaciones	Los reintentos estarán limitados a operaciones idempotentes y se utilizarán timeouts e identificadores de idempotencia para evitar operaciones duplicadas.

Tabla 1. Comparación funcional

Criterio	**Odoo Inventory**	**Zoho Inventory**	**ERPNext**
Tipo	Núcleo de código abierto (Community) y edición comercial en la nube (Enterprise) [4]	SaaS comercial [5]	Código abierto; hosting opcional en Frappe Cloud [14]
Multibodega y traslados	Sí, con bodegas, ubicaciones y rutas de reabastecimiento (no verificado en detalle en esta revisión)	Sí, con órdenes de traslado, estado "en tránsito" y aprobación opcional [6]	Sí, con *Stock Entry* de tipo *Material Transfer* y bodega de tránsito [10]
Lote, vencimiento y FEFO	Sí; FEFO automático por fecha de retiro [1]	Lotes con fecha de vencimiento desde el plan Premium [5]; FEFO asistido (ordena, no asigna) [8]	Lotes con vencimiento [11]; selección automática de lotes en salidas según un criterio configurable en *Stock Settings* (no se confirmó en la documentación oficial si incluye el vencimiento).
Compra y recepción contra la orden	Sí (flujo estándar Compras e Inventario)	Sí (*purchase receives*) [5]	Sí (*Purchase Order* y *Purchase Receipt*)
Devoluciones	Cliente: transferencia inversa [3]. Proveedor:	Cliente: *sales returns* con lote [9]. Proveedor: no se	Cliente y proveedor, contra el documento original [13]

	mismo mecanismo sobre la recepción (no confirmado en documentación oficial)	encontró un documento nativo de devolución (sin confirmar)	
Alertas de stock mínimo	Reglas min/max que generan o sugieren órdenes de compra [2]	Punto de reorden con notificación por correo [7]	Nivel de reorden que crea solicitudes de material automáticamente [15]
Reportes y dashboard	Sí	Sí; Zoho Analytics solo en el plan Enterprise [5]	Sí; reportes configurables por rol [16]
Roles y permisos	Sí, grupos de acceso (no verificado en esta revisión)	Roles predefinidos y personalizados, con restricción por bodega [7]	Por rol, por tipo de documento y por etapa del documento [16]
Arquitectura	Arquitectura de tres capas (HTML5/JS, Python, PostgreSQL) con módulos que se instalan dentro de una misma aplicación [A][B]	No se encontró documentación pública sobre su arquitectura	Aplicación construida sobre Frappe Framework (Python y MariaDB en el servidor, API REST generada automáticamente) [C][D]
Precios	Plan gratuito para una sola app (insuficiente para el caso); Standard desde US $31{,}10 y Custom desde U. 46,70 por usuario al mes en pago anual, con descuento promocional el primer año; valores para EE. UU., consultados el 27/09/2026 [4]	Por organización: de US$29 a US$249 al mes, más complementos (estimación propia no confirmada.) [5]	Licencia gratuita; se paga el hosting por cómputo, no por usuario [14]

Experiencia de usuario	Moderna, pero la configuración logística es compleja*	Sencilla y orientada a pymes*	Funcional, con curva de aprendizaje alta*

Fortalezas y debilidades frente al caso AgroInsumos

Odoo.

Su principal fortaleza es el manejo de vencimientos. La estrategia FEFO prioriza los lotes según su fecha de retiro, que se define como un número de días antes del vencimiento. Además, permite registrar fechas de vencimiento, consumo preferente, retiro y alerta por lote. Esto encaja bien con agroquímicos y semillas. También resuelve la reposición: cuando el stock cae al mínimo, genera o sugiere una orden de compra hasta el máximo. La debilidad es el costo y la complejidad. El plan Custom es el requerido para personalizar mediante Studio, desarrollos propios o la API, y la configuración de rutas multibodega exige una implementación especializada.

Zoho Inventory.

Destaca por su facilidad de adopción, sus traslados con aprobación y la restricción de bodegas por usuario. Con restricciones activas, los usuarios limitados solo ven en el dashboard y los reportes la información de sus bodegas permitidas. Para AgroInsumos presenta tres debilidades:

- **Costo por volumen.** El plan Plus incluye 10 usuarios y 6 ubicaciones [5]. Para 25 usuarios habría que sumar 15 usuarios adicionales a US$7,5 cada uno, lo que da una estimación propia de unos US$240 al mes.
- **FEFO no automático.** Al seleccionar lotes en la factura, estos aparecen ordenados del que vence primero al último, pero la selección sigue siendo manual.
- **Traslados simples.** Solo permite traslados de una bodega a otra, no hacia varias bodegas a la vez.

ERPNext.

Su licencia es libre y sus costos no dependen del número de usuarios. Frappe declara que el producto es 100% de código abierto desde 2009. Tiene tres fortalezas relevantes para el caso:

- **Traslados con tránsito.** Un traslado puede marcarse "en tránsito" y cerrarse después en la bodega destino.
- **Selección por vencimiento.** En la lista de picking, para artículos con lote se elige la bodega cuyo lote está más próximo a vencer.
- **Reorden automático.** Al llegar al nivel de reorden se crea automáticamente una solicitud de material, de compra o de traslado.

Sus debilidades son la curva de aprendizaje y la necesidad de un aliado de implementación, ya que Frappe no presta servicios de implementación directamente. Tampoco pudimos confirmar si el FEFO se aplica de forma automática fuera de la lista de picking.

Conclusiones

1. **Funcionalidades comunes.** Los tres sistemas ofrecen multibodega con traslados, lotes con vencimiento, compras con recepción, devoluciones de clientes, alertas de reorden y roles. Este conjunto es el mínimo aceptable para nuestro sistema.
2. **Requisitos para nuestro sistema.** El sistema debe incluir cinco elementos:
    - FEFO automático, no solo asistido.
    - Traslados con estado "en tránsito" y aprobación.
    - Recepción parcial contra la orden de compra, con captura obligatoria de lote y vencimiento.
    - Devoluciones a proveedor como documento propio, un punto débil en al menos una de las soluciones.
    - Permisos por rol combinados con restricción por bodega, para los ocho roles definidos.
3. **Arquitectura.** Ninguna solución con información pública usa microservicios; Odoo y ERPNext son monolitos modulares. Nuestra propuesta puede aprovechar esa diferencia para escalar de forma independiente servicios como inventario, compras y alertas.
4. **Oportunidades para una distribuidora agrícola regional.** Identificamos cuatro:
    - Alertas de "próximo a vencer" por bodega, con sugerencia automática de traslado hacia la bodega de mayor rotación.
    - Trazabilidad lote-cliente para retiros de producto.
    - Atributos propios del sector, como el registro ICA y la categoría toxicológica.
    - Operación móvil tolerante a mala conectividad en bodegas rurales, además de costos predecibles para una pyme.

Justificación de la arquitectura

Para el sistema de gestión de inventarios de AgroInsumos Cafeteros Ltda. se evaluaron tres alternativas:

- **Monolito tradicional:** una sola aplicación y una sola base de datos, sin fronteras internas definidas.
- **Monolito modular:** una sola aplicación desplegable, organizada en módulos por dominio con interfaces internas explícitas.
- **Microservicios:** servicios desplegados de forma independiente, cada uno con su propia base de datos, comunicados mediante REST síncrono a través de un API Gateway.

La Tabla 1 resume la comparación aplicada al caso.

Tabla 1. Comparación de alternativas arquitectónicas para AgroInsumos Cafeteros Ltda.

Criterio	Monolito tradicional	Monolito modular	Microservicios
Escalabilidad	Suficiente para ~25 usuarios, incluso cuando los traslados se triplican en temporada.	Igual de suficiente.	Permite escalar componentes por separado, aunque el volumen actual no lo exige. Es una ventaja a futuro, no una necesidad actual.
Mantenibilidad	Las reglas de lotes y vencimientos quedan mezcladas con compras y reportes; el código se degrada con el tiempo.	Las fronteras entre módulos existen, pero nada impide cruzarlas bajo presión de entrega.	Las fronteras quedan forzadas por la red y por bases de datos separadas; ningún componente puede leer directamente las tablas de lotes de otro.
Despliegue independiente	Con varias réplicas en Kubernetes se actualiza sin interrupción, pero cada cambio exige reconstruir y probar todo el sistema.	Igual que el tradicional.	Un cambio en reportes solo reconstruye y prueba ese servicio. La ventaja es el menor alcance del cambio, no la ausencia de interrupción.
Tolerancia a fallos	Una consulta pesada del dashboard o un error de memoria afecta al proceso que también atiende despachos.	Igual: módulos distintos, mismo proceso y misma base de datos.	Si reportes o alertas fallan, no se detienen los despachos. Pero las operaciones con dependencias encadenadas son menos disponibles (sección 3).

Organización del equipo	Adecuada para dos desarrolladores.	La más adecuada para dos desarrolladores.	Sin ventaja organizacional con dos personas; exige dominar tres frameworks.
Complejidad operativa	Baja. Un traslado cabe en una transacción ACID.	Baja.	Alta: varias bases de datos, Gateway propio, orquestación y consistencia entre servicios.
Costo	El menor: un servidor virtual.	El menor.	El mayor: En el entorno académico se usará minikube con un nodo; en producción se requerirían al menos dos.

1. Criterios de elección y descarte del monolito modular

Fowler (2015) recomienda iniciar con un monolito, porque los sistemas que arrancan con microservicios suelen fallar al definir mal sus fronteras antes de conocer el dominio. El monolito modular sería una opción viable y más económica para esta empresa, por lo que su descarte requiere argumentos específicos. Se eligieron microservicios por cuatro razones.

Aislamiento de la carga analítica. En ambos monolitos, los reportes de rotación por bodega, lote y proveedor compiten por la misma base de datos que registra salidas y traslados. Esto es más crítico en temporada, que es precisamente cuando la gerencia más consulta el dashboard.

Aislamiento del dominio de lotes y vencimientos. La trazabilidad de agroquímicos y semillas es el dominio con reglas más delicadas del sistema. Tener base de datos propia impide que otros componentes modifiquen esos datos sin pasar por sus reglas. En un monolito modular, esta garantía depende solo de la disciplina del equipo.

Menor impacto de fallas en funciones no críticas. Reportes, dashboard y alertas pueden caer sin detener la operación de bodega. Esto se cumple siempre que las operaciones críticas no dependan de ellos en cadena.

Preparación para el crecimiento funcional. Integraciones previsibles, como la facturación electrónica o un canal de venta en línea, podrían incorporarse como nuevos servicios sin modificar el núcleo de inventario.

Para acotar el desacuerdo con Fowler, las fronteras se trazarán sobre dominios que ya son estables en la operación actual de la empresa, y el número de servicios será reducido. En contrapartida, se aceptan costos concretos:

- Mayor carga operativa para dos desarrolladores.
- Pérdida de transacciones ACID entre dominios.
- Mayor costo de infraestructura.
- Depuración más difícil.

Estos costos se mitigan de tres maneras. Primero, estandarizando la estructura de los servicios (Dockerfile, formato de errores, variables de entorno). Segundo, propagando un identificador de correlación desde el Gateway para rastrear cada petición. Tercero, definiendo health checks en Kubernetes.

2. Consistencia en los traslados

Un traslado tiene un estado intermedio real: la mercancía viaja durante horas entre bodegas. Por eso se modela con los estados solicitado → en tránsito → recibido (o rechazado), un requisito que existiría en cualquier arquitectura. En microservicios, este modelo resuelve además la ausencia de transacciones distribuidas, siguiendo la idea de secuencias de pasos locales con acciones compensatorias (Richardson, 2018):

- Al despachar, el stock se descuenta en origen.
- Al confirmar la recepción, se suma en destino.
- Si el segundo paso falla, el traslado permanece en tránsito y puede reintentarse o revertirse, sin dejar el inventario inconsistente.

3. Riesgos de la comunicación REST síncrona

Disponibilidad encadenada. Si una operación depende de tres componentes con 99 % de disponibilidad cada uno, su disponibilidad es $0{,}99^3 \approx 97$ %. Esto equivale a unas 22 horas mensuales de posible indisponibilidad, frente a unas 7 horas de un componente aislado. Newman (2021) advierte que las llamadas síncronas encadenadas acoplan la disponibilidad de los servicios. Por ello, las operaciones críticas de bodega se diseñarán con un máximo de dos saltos.

Reintentos en cascada. Si cada nivel de la cadena reintenta tres veces, una sola petición puede multiplicarse hasta $3 \times 3 = 9$ llamadas al último servicio. En temporada, esto sobrecarga justamente al servicio que ya está lento. Para evitarlo se aplican tres medidas:

- Los reintentos se hacen solo en el Gateway y solo sobre operaciones idempotentes: un máximo de dos reintentos, con espera creciente de 200 y 400 ms.
- Las escrituras que deban poder reintentarse, como el registro de un traslado, llevan una clave de idempotencia que las vuelve seguras ante duplicados.

- El timeout del Gateway debe superar la suma de los timeouts internos, incluidos los reintentos. Por ejemplo, con llamadas internas de 2 s, el Gateway espera 5 s. Estos valores son iniciales y se calibrarán con mediciones.

Latencia. Las llamadas entre servicios ocurren dentro del clúster y agregan milisegundos. La lentitud que sufre la bodega de Riosucio está en el tramo entre el cliente y el Gateway, y sería idéntica con cualquier arquitectura. Para esa sede se priorizará respuestas ligeras y paginadas.

Como mejora futura queda la implementación de un cortocircuito (circuit breaker) en el Gateway.

4. Reportes, dashboard y alertas de vencimiento

Consultar varios servicios en cada solicitud de reporte, es decir, componer APIs en tiempo real (Richardson, 2018), sería lento y frágil. En su lugar, el servicio de reportes consultará periódicamente a los demás, por ejemplo cada cinco minutos, y guardará una copia de solo lectura en su propia base de datos MongoDB:

- Pedirá solo los registros modificados desde la última consulta.
- El dashboard, los reportes y las alertas de vencimiento se calcularán sobre esa copia, sin cargar las bases transaccionales.
- Se acepta un desfase de algunos minutos, que la interfaz indicará. Es irrelevante para los vencimientos, que se miden en días.
- Si un servicio no responde, la copia conserva los últimos datos disponibles.

5. Autenticación y roles

El Gateway en Laravel valida la firma y la vigencia del token JWT en cada petición. Luego extrae el identificador y el rol del usuario y los reenvía a los servicios en cabeceras internas. Cada servicio aplica su propia autorización según los ocho roles definidos. Solo el Gateway se expone fuera del clúster, y políticas de red de Kubernetes impiden el acceso directo a los servicios, lo que garantiza que esas cabeceras no puedan falsificarse desde el exterior.

6. Conclusión

Los microservicios no son la opción más económica para una empresa de 45 empleados, y un monolito modular atendería su volumen actual. Se eligen porque resuelven tres problemas concretos de AgroInsumos que el monolito modular solo resolvería con disciplina: la competencia entre la carga analítica y la operación, la protección del dominio de lotes y vencimientos, y el impacto de fallas en funciones secundarias. Además, dejan el sistema preparado para crecer en funcionalidades. Sus costos (operación, consistencia distribuida, disponibilidad encadenada) se reconocen explícitamente y se gestionan con pocos servicios, reintentos acotados, estados intermedios y una copia de lectura para reportes.

Microservicios preliminares.

1. Contextos delimitados (bounded contexts)

El análisis del dominio identifica cinco contextos delimitados, cada uno con un lenguaje y unas reglas propias:

- **Identidad y acceso:** usuarios, roles, autenticación y recuperación de contraseña.
- **Catálogo:** la definición de cada referencia (ficha técnica, categoría, unidad de medida y si se controla por lote).
- **Inventario:** existencias por bodega y lote, movimientos, sugerencia FEFO y traslados.
- **Abastecimiento:** proveedores, órdenes de compra, recepción parcial y devoluciones a proveedor.
- **Analítica:** reportes, dashboard y vigilancia de vencimientos.

2. Microservicios propuestos

Se proponen cinco servicios, uno por contexto, más el API Gateway. No se fragmentan más los dominios, porque su tamaño actual no lo justifica y cada servicio adicional aumenta la carga operativa de un equipo de dos personas.

Servicio	**Razón de independencia**	**Tecnología**	**Justificación de la tecnología**
API Gateway	Punto único de entrada. Enruta las peticiones, valida la firma y vigencia del JWT sin consultar a Identidad y reenvía usuario y rol en cabeceras internas.	Laravel + MySQL	Tecnología definida para el Gateway en el curso.
Identidad	La seguridad cambia por razones distintas a la operación de bodega y se aísla para limitar su superficie de exposición.	Django + MySQL	Django incluye de forma nativa modelo de usuarios, grupos y permisos, cifrado de contraseñas y flujo de recuperación de contraseña.
Catálogo	La definición de producto es estable y la consultan los demás servicios, pero no depende de ellos.	Flask + MongoDB	Agroquímicos (ingrediente activo, registro ICA, categoría toxicológica), semillas (variedad, % de germinación) y herramientas tienen

			fichas técnicas heterogéneas, que un modelo documental representa sin tablas dispersas.
Inventario	Es el núcleo transaccional: toda variación de existencias ocurre aquí.	Django + MySQL	Requiere transacciones ACID y bloqueo de filas para evitar sobregiros por lote. Las consultas FEFO, ordenadas por fecha de vencimiento, son naturales en SQL.
Compras	El ciclo con proveedores evoluciona según la política comercial, no según la operación física de bodega.	Django + MySQL	Es un dominio relacional (orden de compra → líneas → recepciones) que exige integridad referencial.
Reportes	Consolida lecturas sin interferir con la operación.	Flask + MongoDB	Cada cinco minutos consulta a los demás servicios y guarda una copia desnormalizada de solo lectura, adecuada para las agregaciones del dashboard.

Alternativa evaluada. Se evaluó fusionar Identidad con el API Gateway para reducir el número de despliegues. Se descartó para mantener el Gateway como punto de entrada sin lógica de dominio, limitado al enrutamiento y la validación del token, y porque las funciones nativas de Django reducen el costo de mantener Identidad como servicio independiente.

3. Ubicación de conceptos compartidos

- **Proveedores → Compras.** El Catálogo describe qué es un producto; a quién se le compra, a qué precio y en qué condiciones es conocimiento comercial. El Catálogo solo guarda el identificador del proveedor habitual, lo que permite filtrar productos por proveedor (RF12) sin consultar a Compras.

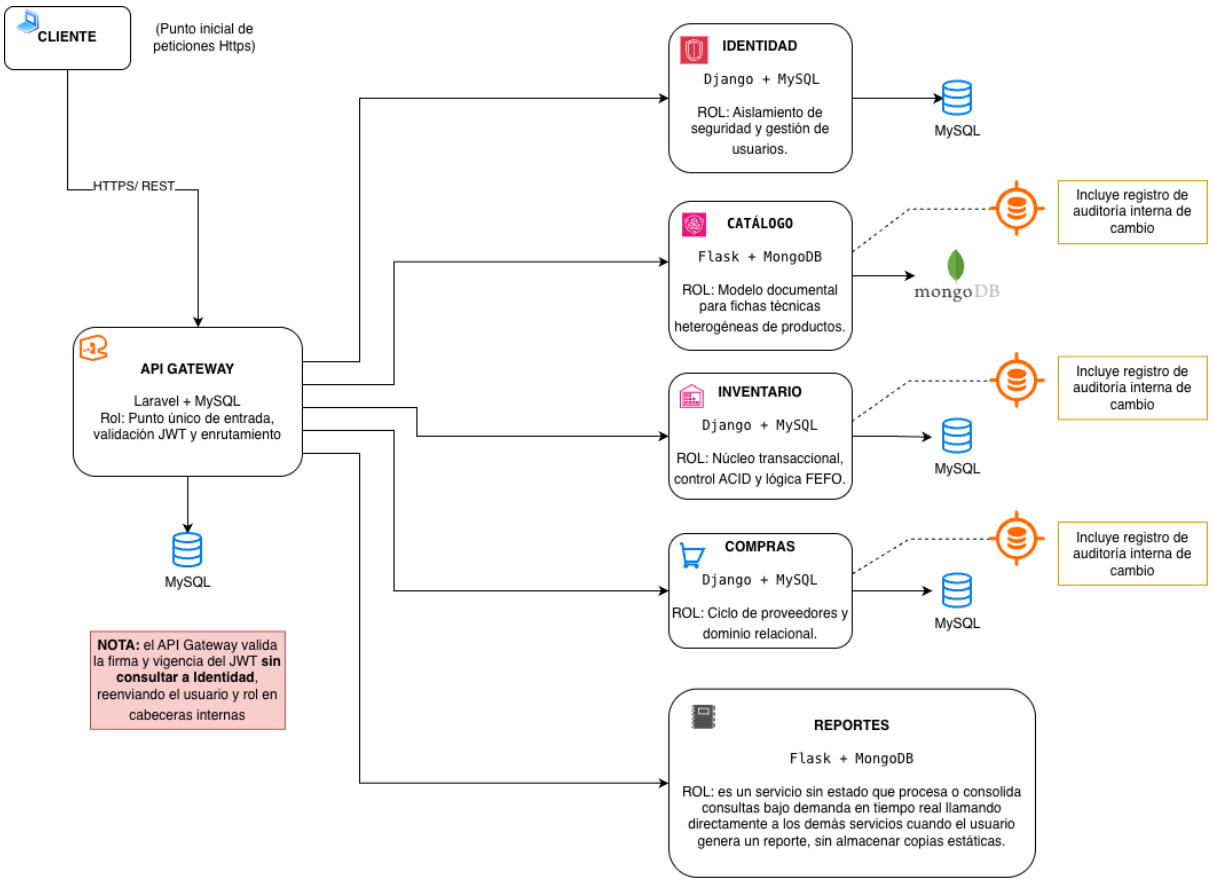
- **Traslados → Inventario.** Despachar descuenta existencias en origen, recibir las suma en destino y rechazar las devuelve al origen. Cada cambio de estado debe ser atómico con el movimiento de stock; separarlo exigiría una transacción distribuida sobre REST.
- **Devoluciones:** se dividen por contexto.
    - *A proveedor → Compras*, porque referencia la orden de compra y el lote recibido; Compras invoca a Inventario para registrar la salida.
    - *De cliente → Inventario*, porque es una entrada al lote original y el alcance no incluye un contexto de ventas.
- **Alertas:** se dividen según su naturaleza.
    - *Stock mínimo → Inventario*, porque se evalúa en cada movimiento y el umbral por bodega y producto pertenece a ese contexto.
    - *Vencimiento → Reportes*, porque depende del paso del tiempo y encaja con la consulta periódica. Un desfase de minutos es irrelevante para plazos que se miden en días.
    - El dashboard presenta ambas alertas de forma consolidada.

4. Comunicación en operaciones críticas

- **Despachar traslado:** Gateway → Inventario (1 salto). Inventario resuelve localmente bodegas, lotes, sugerencia FEFO y el cambio a "en tránsito", sin consultar al Catálogo, porque guarda la referencia al producto.
- **Recibir orden de compra:** Gateway → Compras → Inventario (2 saltos). Compras registra la recepción parcial e invoca a Inventario para crear la entrada por lote con su vencimiento. Si Inventario falla, Compras revierte su transacción local. El identificador de la recepción actúa como clave de idempotencia, para que un reintento no duplique la entrada.
- **Devolver a proveedor:** Gateway → Compras → Inventario (2 saltos), con el mismo esquema.

El Catálogo solo se consulta al crear órdenes de compra, una operación no crítica, y Reportes consulta a los demás servicios fuera de la ruta crítica. Así se respeta en todos los casos el límite de dos saltos.

Arquitectura de Microservicios.



Contratos de API (OpenAPI 3.0)

Servicio	Contrato	Stack	Puerto local
API Gateway	(enrutamiento, ver tabla abajo)	Laravel + MySQL	8000
Identidad	identidad.json	Django + MySQL	8001
Catálogo	catalogo.json	Django + MongoDB	8002
Inventario	inventario.json	Django + MySQL	8003
Compras	compras.json	Django + MySQL	8004
Reportes	reportes.json	Django	8005

Convenciones comunes

- **Prefijo y versión:** /api/v1.
- **Autenticación:** JWT Authorization: Bearer <token> emitido por Identidad. El Gateway valida firma y vigencia (sin llamar a Identidad) y consulta su lista de jti revocados (logout).
- **Cabeceras internas que el Gateway reenvía a los servicios:**
    - X-User-Id, X-User-Role, X-User-Warehouses (ids separados por coma; vacío = todas las bodegas).
    - X-Correlation-Id (lo genera el Gateway si no viene; se propaga en llamadas entre servicios).
- Idempotency-Key en escrituras reintentables (traslados, movimientos, recepciones). Mismo key = misma respuesta.
- **Error estándar:** { "code", "message", "details", "correlationId" }.
- **Paginación:** page (desde 1), size (máx. 100); respuesta { page, size, total, items }.
- updatedSince en listados que Reportes sondea cada 5 min (Catálogo, Inventario, Compras).
- GET /health en cada servicio (liveness/readiness de Kubernetes).
- x-roles en cada operación indica los roles permitidos: ADMIN, GERENTE, JEFE_COMPRAS, AUX_COMPRAS, JEFE_BODEGA, BODEGUERO, VENDEDOR, AUDITOR (ALL = cualquiera autenticado).
- Bajas lógicas (active=false) en productos, proveedores y usuarios.

Enrutamiento del API Gateway

Prefijo externo /api/v1/...

Servicio	**Rutas / endpoints**
Identidad	auth/*, users*, roles
Catálogo	products*, categories*

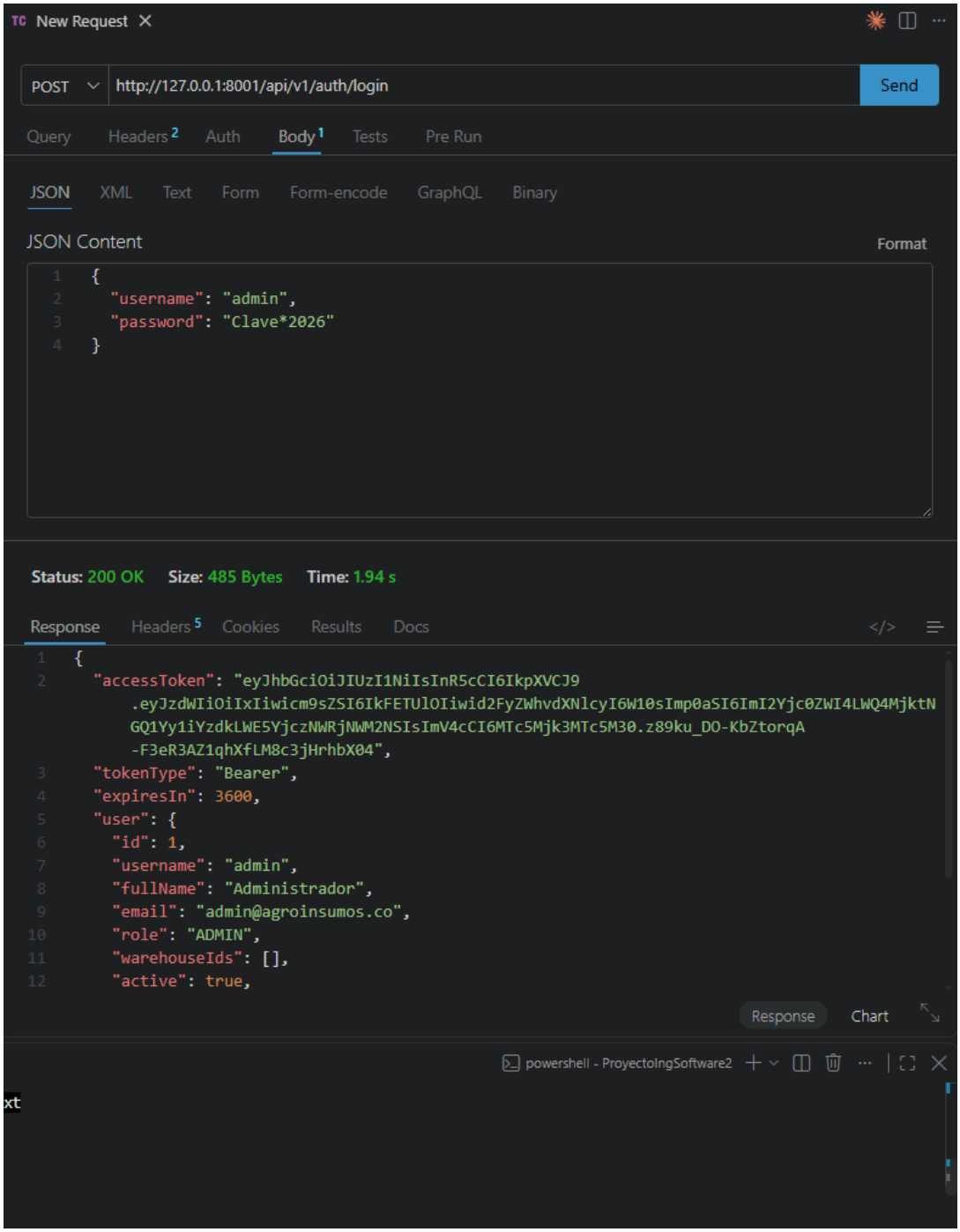
Inventario	warehouses*, stock*, fefo-suggestion, movements*, transfers*, min-stock, alerts/low-stock, customer-returns*, lots/*
Compras	suppliers*, purchase-orders*, supplier-returns*, settings/approval-threshold
Reportes	dashboard, reports/*, alerts/expiry, sync/*
Bloqueado	internal/* — solo tráfico entre servicios dentro del clúster

Comunicación entre servicios (máx. 2 saltos en operaciones críticas)

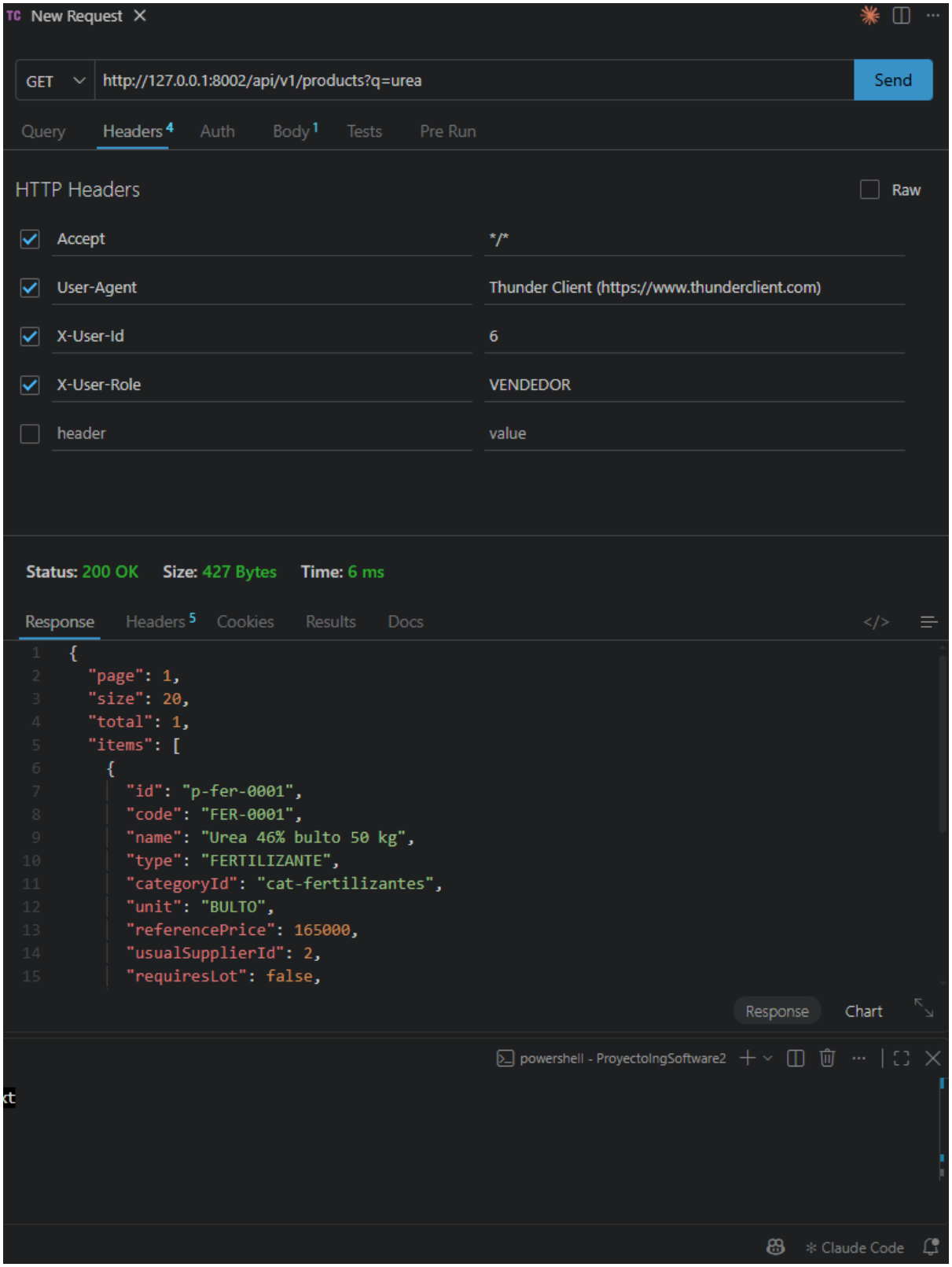
- Despachar traslado: Gateway → Inventario.
- Recibir OC: Gateway → Compras → Inventario POST /internal/entries (Idempotency-Key = id de recepción).
- Devolver a proveedor: Gateway → Compras → Inventario POST /internal/exits.
- Crear OC: Compras → Catálogo GET /products/{id} (no crítica).
- Reportes → Catálogo/Inventario/Compras con updatedSince cada 5 min (fuera de la ruta crítica).

Pruebas en ThunderClient

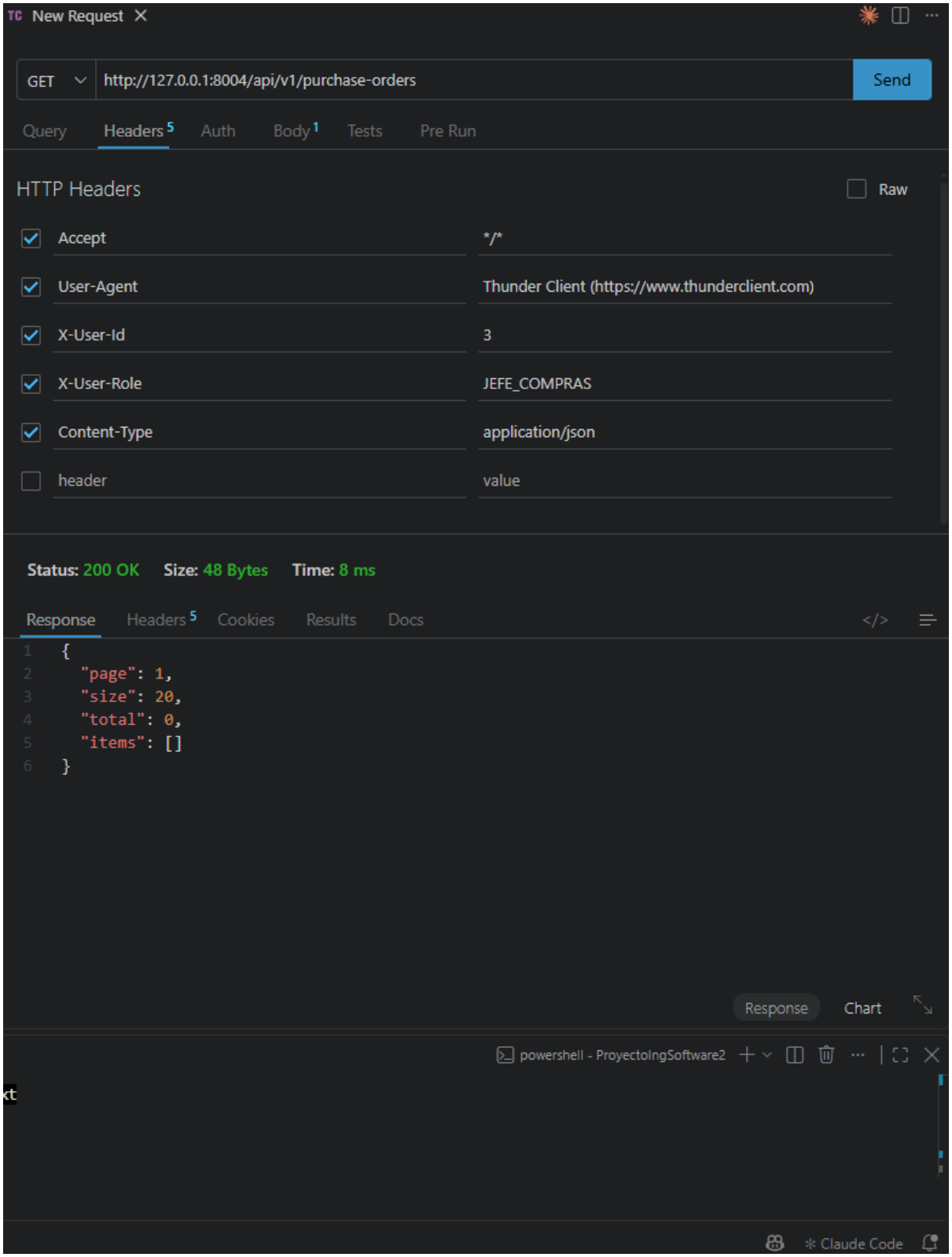
- Request POST para realizar login:



- Request GET de producto con Headers



- Request GET de orden de compra con headers



- Request GET de productos sin header de users

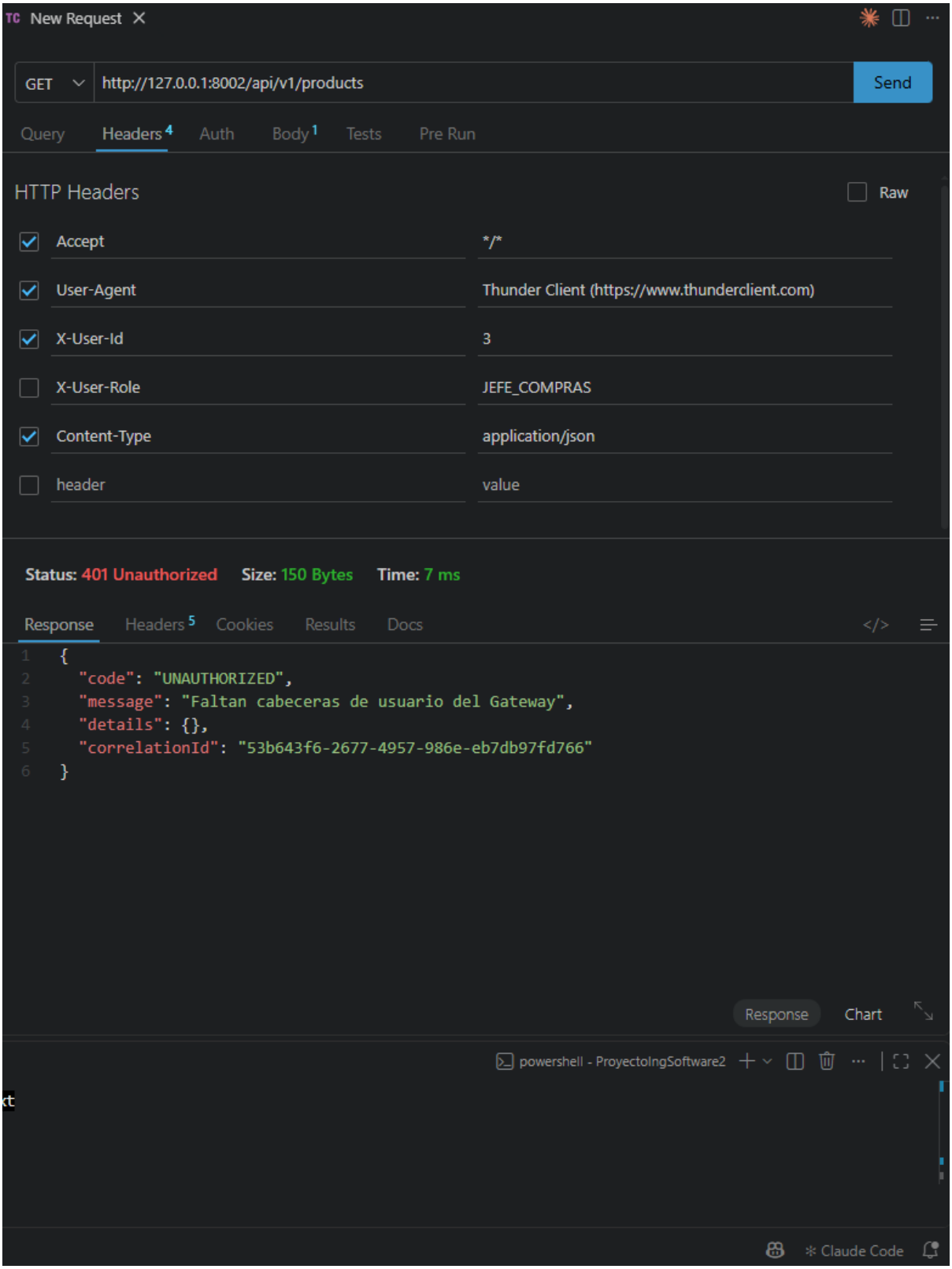
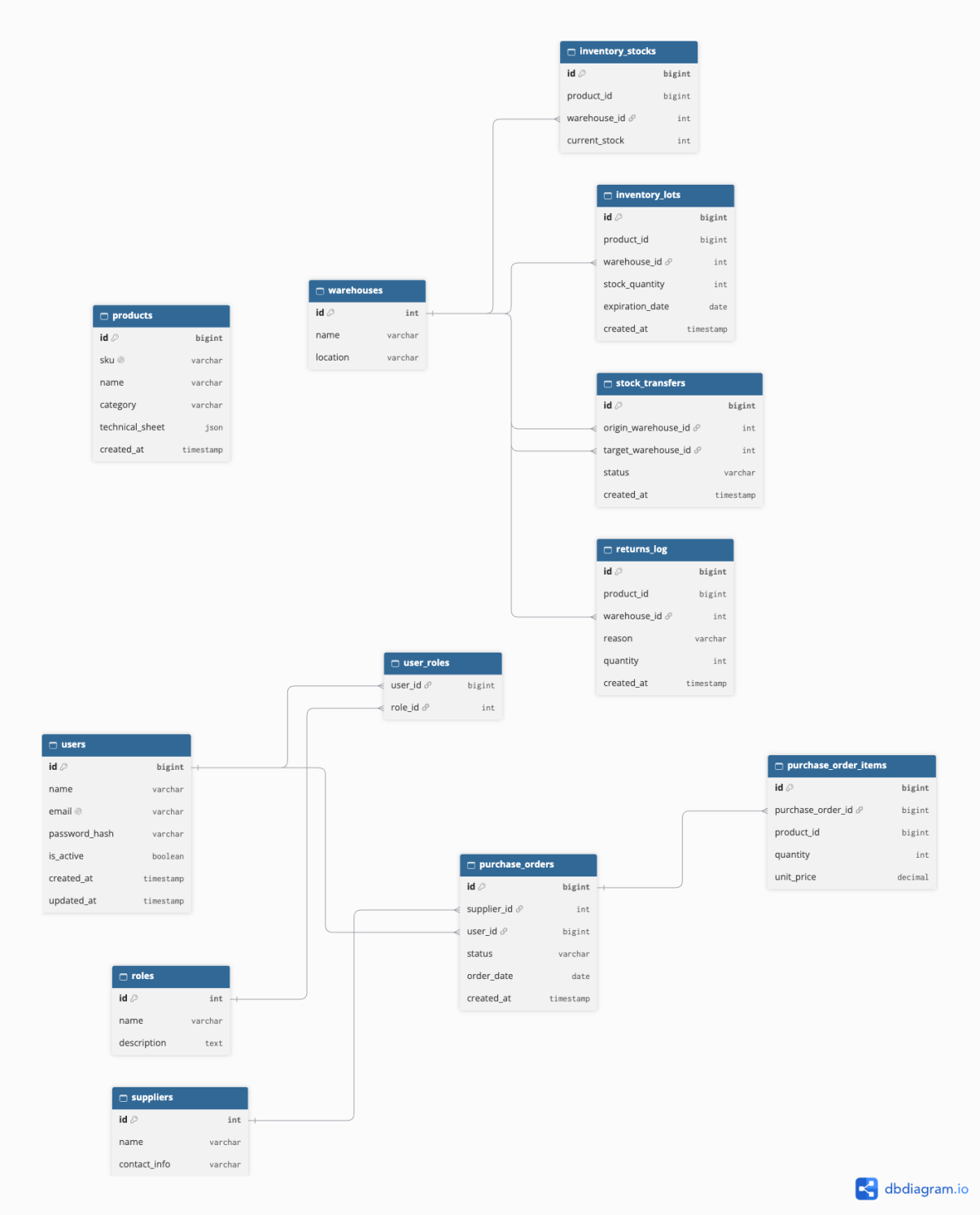


Diagrama Entidad-Relación.



Bitácora

Proyecto	Sistema de Gestión de Inventarios - AgroInsumos Cafeteros Ltda.
Equipo/ Integrantes	

Entrega N.	1
Periodo Cubierto	25/09/2026 - 25/10/2026

Fecha	**Integrante**	**Herramienta de IA**	**Tarea / actividad apoyada**	**Prompt utilizado (resumen)**	**Nivel de Intervención Humana**	**Resultado / aprendizaje**
27/09/2026	Sebastián	Claude	Selección de la empresa caso de estudio	Se pidieron 3 empresas colombianas ficticias con varias bodegas, traslados, proveedores, órdenes de compra y devoluciones, con justificación para microservicios y debilidades de cada una.	Medio	Se eligió AgroInsumos Cafeteros por su contexto regional (Eje Cafetero), sus 5 sedes con traslados frecuentes y el control por lote y vencimiento, que justifican separar dominios. Antes de usar la propuesta se verificó en el enunciado que la empresa puede ser ficticia, pues surgió la duda de si el objetivo 1 lo impedía: ese objetivo se refiere a los sistemas del mercado, no a la empresa.
27/09/2026	Sebastián	Claude	Definición del	Desarrollar la	Medio	Se usó la estructura y

			problema y alcance del sistema	empresa elegida (sedes, productos, usuarios y roles, manejo actual del inventario, problemas concretos) y definir qué incluye y qué excluye el sistema.		la mayoría del contenido. Se contrastó el alcance con los RF01–RF14 del enunciado y se agregaron los que faltaban: dashboard, reporte de valorización y rotación, y recuperación de contraseña. Aprendizaje: la IA propone alcances completos pero no los valida contra los requisitos del docente; hay que revisarlos uno por uno.
27/09/2026	Sebastián	Claude	Justificación técnica de microservicios vs monolito	Comparar monolito vs microservicios para AgroInsumos con criterios explícitos, contras, riesgos de REST síncrono y conclusión. Luego se pidió a la IA criticar su propia respuesta	Alto	La primera versión tenía errores conceptuales detectados en la autocrítica: confundía la latencia de red de la sede con la latencia interna del clúster, atribuía a los microservici

				y reescribirla con correcciones.		os ventajas que también tiene un monolito en Kubernetes (despliegue sin caída), proponía replicar datos sin mecanismo de sincronización y no consideraba el monolito modular. Se rechazó el enfoque de "arquitectura impuesta" y se reescribió con argumentos defendibles, costos reconocidos y fuentes citadas. Aprendizaje: la IA tiende a sobrevender la arquitectura con argumentos genéricos, y pedirle que se critique a sí misma revela sus errores.
27/09/2026	Sebastián	Claude	Investigación comparativa de Odoo Inventory, Zoho Inventory y	Comparar los tres sistemas en 11 criterios ligados al caso, con fortalezas,	Medio	La IA marcó como no verificados varios datos (devoluciones a proveedor, permisos en

			ERPNext	debilidades, conclusiones y fuentes oficiales; luego pedir a la IA que marcara los datos dudosos		Odoo, precios de Zoho) y reconoció que la arquitectura de Zoho no es pública. Se detectaron citas sueltas, precios dependientes del país del visitante y conclusiones que contradecían la justificación (escalabilidad) y el alcance (FEFO automático, oportunidades excluidas), y se corrigieron. Aprendizaje: la IA mezcla datos verificados con suposiciones, y sus conclusiones deben cruzarse con el resto del documento para mantener la coherencia.
28/09/2026	Sebastián	Claude	Identificación preliminar	Identificar bounded contexts y	Alto	Se usó la identificación de los

			de microservicios	proponer máximo 6 servicios coherentes con la justificación (REST síncrono, dos saltos máximo, copia de lectura para reportes), y luego evaluar si algún servicio sobraba o mezclaba dominios.		cinco contextos y la ubicación de conceptos compartidos. En su autocrítica, la IA recomendó fusionar Identidad con el Gateway por el tamaño del equipo; se descartó porque el enunciado sitúa la gestión de usuarios como dominio y limita el Gateway a aspectos transversales, y la fusión se documentó como alternativa evaluada. Se cambiaron las tecnologías de Identidad y Compras de Laravel a Django, para dejar Laravel solo en el Gateway como se trabajó en clase y aprovechar la autenticación nativa de

						Django. Se corrigieron la frecuencia de sondeo de Reportes, para alinearla con la justificación, y el filtro de productos por proveedor (RF12), que quedaba sin resolver al ubicar proveedores en Compras. Aprendizaje: una recomendación técnicamente razonable de la IA puede chocar con los requisitos del enunciado o con decisiones previas del documento, por lo que cada propuesta debe contrastarse con ambos.
28/09/2026	Byron	OpenAI	Creación de la sección de explicación del repositorio	dado el siguiente link de repositorio de github: JeanByron	Alto	Se recordó la importancia y utilidad de github como control de

				/ProyectoIngSoftware2.git, prepara una presentación de dicho respositorio y cómo su funcionalidad será el control de versiones del proyecto		versiones
28/09/2026	Byron	OpenAI	Formalización de los requerimientos funcionales según los resultados obtenidos del análisis	Se solicitó analizar el problema y alcance definido para el sistema de gestión de inventarios de AgroInsumos Cafeteros Ltda. y, a partir de las necesidades identificadas, establecer los requerimientos funcionales y no funcionales del sistema.	Medio	Se obtuvo el alcance tecnológico del proyecto
22/10/2026	Sebastián	Gemini	Confirmación Conexión Microservicios.	Se solicita verificar la arquitectura de microservicios	Alta.	Se corrigió conexiones que no eran necesarias, se simplificó la

				realizada para el proyecto.		arquitectura.
22/10/2026	Sebastián	Gemini	Diagrama Entidad-Relación	Verifica si el diseño realizado para el diagrama entidad-relación de las bases de datos está completa y corrige errores que veas.	Media	Se corrigieron Tablas, conexiones entre ellas y se aprendieron las referencias lógicas de la tabla productos.
25/10/2026	Byron	Claude	Claude Contexto del proyecto	Generar una skill con el contexto de la Entrega 1 y del enunciado, actualizable con cada avance	Medio	Contexto reutilizable. Detectó que el informe y el enunciado numeran los RF distinto (el "RF12" del filtro por proveedor).
25/10/2026	Byron	Claude	Plan de la Entrega 2	Proponer cómo desarrollar contratos, pruebas en Thunder y bitácora, y esperar confirmación	Medio	Plan aprobado. Se descartó el mock en Node (Prism) por la regla de usar solo tecnologías de clase.
25/10/2026	Byron	Claude	Revisión de alineación	Verificar el plan contra el informe de la Entrega 1	Alto	Faltaban 8 puntos (updatedSince, health checks, restricción por bodega, umbral de aprobación, flag de lote,

						entre otros). Se decidió logout con lista negra y sin refresh token. Aprendizaje: contrastar siempre con las decisiones previas del documento.
25/10/2026	Byron	Claude	Contratos OpenAPI	Diseñar los 5 contratos con convenciones comunes	Medio	5 contratos válidos. El primer borrador tenía 8 errores de sintaxis (comas sin comillas) que solo detectó el validador.
25/10/2026	Byron	Claude	Servicios de prueba y Thunder	Crear servicios mínimos en localhost y una colección GET/POST	Medio	52/52 peticiones OK. Se verificaron idempotencia en la recepción, y diferencias en traslados. Se detectó que localhost en Windows añade ~2 s por petición.

Reflexión final.

El desarrollo de este proyecto permitió comprender que la construcción de un sistema de información no consiste únicamente en implementar funcionalidades, sino en analizar previamente las necesidades reales de una organización y traducirlas en una solución técnica coherente. En el caso de **AgroInsumos Cafeteros Ltda.**, el análisis de los procesos actuales permitió identificar problemas asociados al manejo manual de inventarios, la falta de

trazabilidad, el control de lotes y vencimientos, los traslados entre bodegas y la gestión de compras y devoluciones.

La investigación de sistemas existentes permitió reconocer funcionalidades que son comunes en las soluciones de gestión de inventarios, pero también identificar necesidades específicas del caso de estudio que requieren un tratamiento particular, como el control FEFO, los estados de los traslados, la recepción parcial de órdenes de compra y la trazabilidad de lotes. Esto permitió que los requerimientos propuestos no fueran definidos únicamente desde una perspectiva tecnológica, sino a partir de los problemas concretos identificados en la empresa.

Asimismo, la definición de los requerimientos funcionales y no funcionales permitió establecer con mayor claridad qué debe hacer el sistema y bajo qué condiciones debe operar. Esta etapa resulta fundamental porque sirve como punto de referencia para las decisiones posteriores de arquitectura, diseño e implementación. El alcance también permitió diferenciar las funcionalidades prioritarias de aquellas que quedan fuera del proyecto, evitando ampliar innecesariamente la solución.

La decisión de utilizar una arquitectura basada en microservicios representa un reto adicional para un equipo pequeño, debido a la mayor complejidad de despliegue, comunicación y operación frente a un monolito modular. Sin embargo, el análisis permitió justificar esta elección principalmente por la necesidad de separar dominios como inventario, compras, identidad y reportes, así como por la posibilidad de aislar cargas analíticas y evolucionar los componentes de manera independiente. Esta decisión demuestra que una arquitectura debe seleccionarse en función de las características y necesidades del sistema, y no simplemente por adoptar una tecnología considerada moderna.

Referencias.

[A] Documentación oficial para desarrolladores (arquitectura)
Odoo. (s. f.). *Chapter 1: Architecture Overview*. Odoo 18.0 Documentation. https://www.odoo.com/documentation/18.0/developer/tutorials/server_framework_101/01_architecture.html
[B] Repositorio oficial
Odoo S.A. (2026). *odoo/odoo* [Repositorio de código fuente]. GitHub. https://github.com/odoo/odoo
[C] Repositorio oficial de Frappe Framework, la base de ERPNext
Frappe Technologies. (2026). *frappe/frappe* [Repositorio de código fuente]. GitHub. https://github.com/frappe/frappe
[D] Repositorio oficial de ERPNext
Frappe Technologies. (2026). *frappe/erpnext* [Repositorio de código fuente]. GitHub. https://github.com/frappe/erpnext
[1] Odoo, "FEFO removal", Odoo 18.0 Documentation. https://www.odoo.com/documentation/18.0/applications/inventory_and_mrp/inventory/shipping_receiving/removal_strategies/fefo.html
[2] Odoo, "Replenishment", Odoo 18.0 Documentation. https://www.odoo.com/documentation/18.0/applications/inventory_and_mrp/inventory/warehouses_storage/replenishment.html

[3] Odoo, "Returns and refunds", Odoo 18.0 Documentation. https://www.odoo.com/documentation/18.0/applications/sales/sales/products_prices/returns.html
[4] Odoo, "Pricing". https://www.odoo.com/pricing
[5] Zoho, "Inventory Plans and Pricing". https://www.zoho.com/us/inventory/pricing/
[6] Zoho, "Transfer Orders – User Guide". https://www.zoho.com/us/inventory/help/warehouses/transfer-orders.html
[7] Zoho, "Multiple Warehouses", "Users & Roles" y "More Actions on Items" – User Guide. https://www.zoho.com/us/inventory/help/warehouses/warehouses-overview.html · https://www.zoho.com/us/inventory/help/settings/users.html · https://www.zoho.com/us/inventory/help/items/item-managing.html
[8] Zoho, "Tracking Expiry Dates" y "Batch Tracking". https://www.zoho.com/de-de/inventory/kb/advanced-inventory-tracking/first-expiring-batch.html · https://www.zoho.com/us/inventory/help/advanced-inventory-tracking/batch-tracking.html
[9] Zoho, "Sales Returns – User Guide". https://www.zoho.com/us/inventory/help/sales-returns/sales-returns-overview.html
[10] Frappe, "Stock Entry", ERPNext Documentation. https://docs.frappe.io/erpnext/user/manual/en/stock-entry
[11] Frappe, "Batch", ERPNext Documentation. https://docs.frappe.io/erpnext/user/manual/en/batch
[12] Frappe, "Pick List", ERPNext v13 Documentation. https://docs.erpnext.com/docs/v13/user/manual/en/stock/pick-list
[13] Frappe, "Sales Return" y "Purchase Return". https://docs.frappe.io/erpnext/user/manual/en/sales-return · https://docs.frappe.io/erpnext/v12/user/manual/en/stock/purchase-return
[14] Frappe, "ERPNext Pricing" y "Frappe Cloud Pricing". https://frappe.io/erpnext/pricing · https://frappe.io/cloud/pricing
[15] Frappe, "Auto Creation of Material Request". https://docs.erpnext.com/docs/user/manual/en/auto-creation-of-material-request
[16] Frappe, "Role and Role Profile" y "Role Based Permissions". https://docs.frappe.io/erpnext/user/manual/en/role-and-role-profile · https://docs.frappe.io/erpnext/user/manual/en/role-based-permissions

Fowler, M. (2015, 3 de junio). *MonolithFirst*. Blog personal de Martin Fowler. https://martinfowler.com/bliki/MonolithFirst.html

Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems* (2.ª ed.). Publicaciones O'Reilly Media.

Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Editorial Manning Publications.